Mandatory Practical Work 02 – 20/04/2026
Computational graphs and optimizers

Deadline : Monday May 18 ; **Format** : pdf or ipynb. Clearly state the group members'**name** and **surname** at the beginning of the report.

1 Overview

Reminder : the *Mandatory Practical Works* are intended to be done in groups of max 3 students. The goal is to apply the concepts seen in the lectures and practical sessions, and to document your experimentation results in a report. The report should be clear, concise, and well-structured, including all relevant details about your experiments such as the hyperparameters you used, the results you obtained, and any insights or conclusions you can draw.

2 Context

Computational graphs

In this practical work, we will be using a simple computational graph implementation provided in the file `cgnodes.py`.

This implementation follows the definition given in the class : a computational graph is a **directed acyclic graph** where **nodes** correspond to operations or variables and **edges** correspond to links between nodes.

With `cgnodes.py`, the creation of the graph involves three steps :

a) Creation of the value nodes with `ValueNode()` objects.
b) Creation of the operator nodes with objects `MultiplyNode()`, `SquareNode()` ...
c) Build the graph by declaring inputs and outps as 2 lists of `ValueNode()` objects.
d) Use of the graph with forward and backward calls, e.g. `cg.forward(inputs)`

Figure 1 shows an example and the code used to implement the graph is given in listing 1.

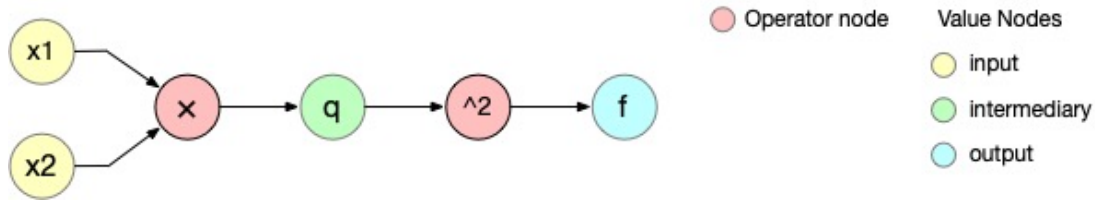

FIGURE 1 – Simple graph computing $f = (x_1 x_2)^2$. The implementation uses operator nodes (red) and three types of value nodes storing scalar float values : input (yellow), intermediary (green) and output (blue).

```
# first create all ValueNode objects
x1 = ValueNode()
x2 = ValueNode()
q = ValueNode()
f = ValueNode()

# second create all <Operator>Node objects
mult = MultiplyNode(x1, x2, q)
square = SquareNode(q, f)

# finally build the graph by declaring inputs and outputs as 2 lists of ValueNode objects
cg = CompGraph([x1, x2], [f])

# test the graph with some random inputs
cg.forward([2.0, 4.0])
print(f"f = {f.v}")  # should print 64.0
```

Listing 1 – Simple computational graph example

Regression problem

We will play with a regression problem to predict a value y from input features x. The simplest regression model is a linear model defined with the parameters θ as follows :

$$\hat{y} = h_\theta(\mathbf{x}) = \theta_0 + \theta_1 x \tag{1}$$

The **loss** typically used for regression problems is the *Mean Square Error* (MSE) defined with :

$$J(\theta) = \frac{1}{2N} \sum_{n=1}^{N} (\hat{y}_n - y_n)^2 \tag{2}$$

As illustrated in the python notebook provided for this practical work (and as alredy seen in practival work week 1), there exist a *closed form* solution to this problem given by the following normal equation :

$$\theta = (X^T X)^{-1} X^T \vec{y} \tag{3}$$

With X and $\vec{y}$ defined as

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & \\ 1 & \vdots \\ 1 & \\ 1 & x_N \end{pmatrix} \tag{4}$$

$$\vec{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \tag{5}$$

The Eq. 3 allows to compute the "best" θ's minimising the squared difference between the gotten outputs and the *target* outputs defined with Eq. 2. However, this implementation does not allow to handle large datasets (with N large, matrix inversion becomes more difficult) and does not allow for incremental learning (fine-tuning) when new data are coming. Therefore we want now to experiment with a **backpropagation** strategy using a **computational graph**.

The graph version of Eq. 1 is illustrated on Figure 2.

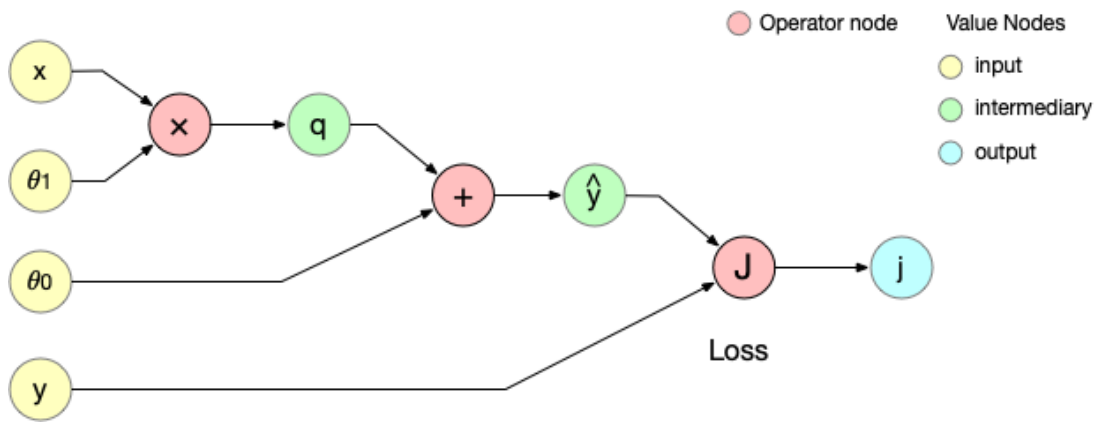

FIGURE 2 – Computational graph corresponding to the linear regression

3 Objectives

It is now time to move to the Python code `cdgnodes` and notebooks `cg-linear-regressopm-stud.ipynb` notebook that you need to complete.

Main objectives

a) Understand the concepts of backpropagation and of forward and backward passes in the context of a computational graph.
b) Implement new nodes in a computational graph.
c) Implement a training loop with stochastic and batch approaches
d) Reimplement and experiment with different optimization algorithms and compare their performance.

Optional objectives

Pick at least 2 of the following optional objectives to further explore and analyze the performance of your models :

— Investigate the use of 2nd order model instead of the simple linear model.
— Re-implement and experiment with more advanced optimizers such as RMSProp, Nesterov or Adam.
— Implement an early stopping strategy in your training loop.
— Implement a *Learning Rate Decay on Plateau* strategy in the training loop.
— Normalize the input data with a zero norm approach and compare to your experiments without normalization.